NAMINGCONTEST.COM

NamingContest.com — Developer Handbook

Technical reference for maintaining and extending the platform

---

Technical reference for maintaining, reviewing, and extending the platform. Written for a senior developer arriving cold. The plain-English companion is `PLATFORM-GUIDE.md`; email-lifecycle testing lives in `TESTING-EMAILS.md`.

No credentials appear in this document. Access is shared privately.

---

1. Product in one paragraph

A paid naming-contest platform. A creator answers a guided brief, pays a one-time fee via Stripe ($9/$19/$39 keyed to voter capacity 10/30/90), and shares one invitation link. Participants join via email magic link, submit names (creator-set cap, 1–5), then vote (up to 3 picks, tallies hidden until close). Phases advance automatically on timers. The creator crowns a winner from a leaderboard; lifecycle emails go out at each transition; there is a public reveal page per contest.

- Production: https://namingcontest.com (soft beta gate, client-side password)
- Hosting: Vercel, project `namingcontest-prototype`
- Supabase project ref: `kgcggyuoezaygyawnlcs`

---

2. Stack

LAYER	TECH	NOTES
Frontend	React 18.3, Vite 5.4, react-router 6	SPA, no SSR
Database	Supabase Postgres	RLS on every table
Server logic	Supabase Edge Functions (Deno)	5 functions, see §7
Auth	Supabase Auth, magic links only	implicit flow, no passwords
Payments	Stripe Payment Intents	in-modal confirmCardPayment, no hosted checkout
Email	Resend (API + SMTP)	plus 2 Supabase Auth templates
Scheduling	pg_cron	phase transitions + rate-limit pruning
Secrets	Supabase secrets + Vault	see §8
PDF export	jsPDF 4 + html-to-image	client-side capture of a hidden node
Analytics	@vercel/analytics	`<Analytics />` in App.jsx

3. Repository layout

```
COMPLETE-FINAL/
├── src/                    # the app (working dir for dev)
│   ├── pages/v4/           # the real product surfaces
│   ├── pages/legal/        # /privacy, /terms (real, keep)
│   ├── pages/system/       # 404, /link-expired, /error, /contact
│   ├── pages/*.jsx         # LEGACY v1 prototype pages (see §12)
│   ├── components/v4/      # product components (AvatarMenu, SignInModal…)
│   ├── data/v4/            # briefQuestions, segmentTheme, voterTiers, mock data
│   ├── lib/                # supabaseClient, AuthContext
│   ├── utils/              # v4Brief (guest blob), v4Anonymity, exports…
│   └── styles/             # v4.css (large), landing-v3.css, mobile.css…
├── supabase/
│   ├── migrations/         # 0001–0022, the full DB history (§6)
│   ├── functions/          # edge functions + _shared/ (email.ts, rateLimit.ts)
│   └── templates/          # copies of the 2 dashboard email templates
├── docs/                   # this file + guides
└── public/                 # favicons, email logo/icons PNGs
```

4. Branch and deploy workflow

- `backend` — working branch. All development lands here.

- `master` — production. **Pushing master auto-deploys to Vercel.**
- Promotion is always a `--no-ff` merge: `git checkout master && git merge --no-ff backend && git push`.
- Tags: `demo-complete-2026-07-20` is the snapshot with the demo surface still routed (see §12); older `v5-live` / `v6-live` / `pre-demo-teardown` predate the backend.
- Vercel also builds preview deployments per branch push (staging).

Verify a deploy landed: fetch `https://namingcontest.com/`, take the hashed `/assets/index-*.js` name, and grep it for a string from your change. Note the SPA rewrite (`vercel.json`) returns HTTP 200 + index.html for *every* path, so status codes prove nothing — check content-type / content.

---

5. Local development

```
npm install
cp .env.example .env       # fill in values (see §8)
npm run dev                # Vite on :5173
npm run build              # production build — run before every promote
```

The build must pass before promoting; there is no CI, the discipline is manual.

---

6. Database

Tables (0001_initial_schema)

`profiles` (1:1 auth.users; display_name, avatar_url — **no email stored**), `contests`, `participants`, `submissions`, `votes`. All RLS-enabled.

Migration catalog

Migrations are applied by pasting into the Supabase SQL editor, named `NNNN_slug`. **There is no migration runner** — the numbered files in `supabase/migrations/` are the source of truth and must be applied in order on a fresh environment.

#	PURPOSE
0001	Schema, RLS policies, submission/vote rule triggers, vote_count sync
0002	Storage bucket `uploads` (public read, per-user-folder writes), avatar_url
0003	RLS recursion fix (security definer helper)
0004	`get_join_info` — public join-page fields RPC
0005	`contest_is_joinable` — fixes 42501 on join
0006	user FK → auth.users
0007	Realtime publication for live dashboard
0008	`cancelled` status
0009	pg_cron `advance-contest-phases` (every minute; submission→voting→closed on deadlines)
0010	join info: creator name + deadlines
0011	`get_winner_info` — public reveal RPC (granted to anon)
0012	`auth_user_id_by_email` (avoids generateLink OTP-cooldown bug)
0013	pg_net trigger → notify function on voting/winner transitions
0014	`notified_voting_at` / `notified_winner_at` dedupe stamps
0015	notify secret moved to Vault
0016	Submission cap honours creator setting, clamped to 5
0017	**Price derived server-side from voter_tier** (trigger overwrites client value)
0018	`get_ballot` — resolved ballot RPC (anonymity + hidden tallies)
0019	`submissions_read` lockdown (own rows + creator only)
0020	Rate-limit table + `rate_limit_take` + hourly prune cron
0021	Payment-column guard trigger (client can't set paid/status/deadlines/creator/tier)
0022	Voter-tier capacity enforced on participant insert
0023	`voting_closed` notification to the creator + `contest_creator_email` + `notified_closed_at`

Server-enforced invariants (the security model)

Everything below is enforced in Postgres, not in the UI:

1. Price = f(voter_tier): 10→$9, 30→$19, 90→$39; unknown → null (unpayable).
2. A contest cannot become paid/live except via the service role (confirm-launch after Stripe verification). Status can only move to `cancelled` from the client.

3. Participant count ≤ voter_tier (null tier = legacy, uncapped; over-capacity contests are grandfathered — next join blocked, nobody removed).
4. Submissions per user ≤ creator's setting, hard ceiling 5.
5. Votes: ≤3, participants only, not for your own name, only while `voting`.
6. Participants never see `user_id` or mid-vote tallies (get_ballot resolves display names per the contest's anonymity mode; counts released when closed, or always to the creator).
7. Votes readable only by their caster. Creator sees aggregates, not ballots.
8. Rate limits: contact 3/h/IP; launch 5/h/IP + 3/h/email. **Fails open** by design (nuisance endpoints; availability > strictness). IPs stored only as date-salted SHA-256 hashes.
9. All user-controlled values escaped in email HTML (`esc()` in `_shared/email.ts`; `bodyHtml` is the one field escaped at call sites — its type comment explains).

---

7. Edge functions

Deploy: `npx supabase functions deploy <name> [--no-verify-jwt] --project-ref kgcggyuoezaygyawnlcs`

FUNCTION	JWT	PURPOSE
`launch-contest`	**no**	Guest launch: find-or-create user by email, insert draft. Rate-limited. Never uses `admin.generateLink` (it trips the per-email OTP cooldown; the app sends the magic link via `signInWithOtp` so it carries the browser's context)
`create-payment-intent`	yes	PI for `contests.price` (server-authoritative), rejects < $0.50
`confirm-launch`	yes	Retrieves PI from Stripe, checks status/metadata/amount, flips contest live, sends branded receipt
`notify`	**no** (secret header)	Lifecycle emails, called by DB trigger via pg_net with `x-notify-secret` (Vault). Dedupe stamps written BEFORE sending. Resend batch endpoint (100/call)
`stripe-webhook`	**no** (signature)	Backstop for `payment_intent.succeeded`. Verifies the Stripe signature with `constructEventAsync` (the sync variant needs Node crypto), then delegates to confirm-launch. Fails closed: with no `STRIPE_WEBHOOK_SECRET` it rejects everything
`contact`	**no**	Contact form → team inbox (reply_to = visitor) + visitor receipt. Rate-limited, escaped, field caps

Shared modules: `_shared/email.ts` (design system + `esc` + `sendEmail` + `FROM`), `_shared/rateLimit.ts` (IP hashing + fail-open take).

8. Environment variables and secrets

Vercel (build-time, inlined into the JS bundle — changing one requires a redeploy, and none of these are secret-safe): `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_STRIPE_PUBLISHABLE_KEY`, `VITE_BETA_PASSWORD` (unset = gate off; it ships in the bundle — obscurity, not access control).

Supabase function secrets (`npx supabase secrets set K=V --project-ref …`): `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `RESEND_API_KEY`, `NOTIFY_SECRET`, `SITE_URL`, `CONTACT_TO` (defaults to hello@namingcontest.com).

Vault: the notify secret also lives in Supabase Vault (0015) for the DB trigger side. Rotating NOTIFY_SECRET means updating both (vault.update_secret + secrets set).

Supabase dashboard, not in repo: Auth SMTP settings (Resend, smtp.resend.com:465, sender noreply@namingcontest.com), the two Auth email templates (Magic Link, Confirm signup — repo copies in `supabase/templates/`, keep in sync by re-pasting), Redirect URLs allow-list.

9. Auth flow

- Magic links only, implicit flow (supabase-js default). `signInWithOtp` with `emailRedirectTo`; new addresses get the **Confirm signup** template, existing ones **Magic Link** — both restyled.
- Rejected links arrive as `#error=…&error_code=otp_expired` on the redirect URL. `AuthContext` parses this and `App` routes to `/link-expired` (links are one-time; mail scanners commonly consume them first).
- Sign-out awaits `auth.signOut()` and falls back to `{scope:'local'}` on failure — being unable to leave is worse than an unrevoked refresh token.
- **The guest blob** `localStorage.v4_contest_setup` (via `utils/v4Brief.js`) carries a guest's identity and in-progress contest before an account exists. Historic bug source: it must NEVER stand in for a real session. Current rules: AuthContext drops its identity fields when a session appears under a different email (brief answers are kept — that's the guest-to-user upgrade path); real contest pages never fall back to it; Settings guards against signed-out visitors unless the blob holds a genuinely-theirs contestId. **Review focus: any new `readSetup()` fallback must be gated on the absence of a session.**

10. Payment flow

1. Review page → `launch-contest` (guest) or direct insert (signed-in): draft row, `paid=false`. Trigger 0017 overwrites any client `price`.
2. `create-payment-intent` reads `contests.price` server-side → PI with `metadata.contestId`.
3. Browser confirms card in-modal (`stripe.confirmCardPayment`).

4. `confirm-launch` retrieves the PI (expand latest_charge), verifies `status === 'succeeded'`, `metadata.contestId`, `amount === price*100` (valid because price is server-authoritative), then service-role-updates: `paid`, `status='submission'`, `launched_at`, both phase deadlines — the columns trigger 0021 blocks from every non-service-role path. Sends the receipt email with Stripe's `receipt_url`.

5. `stripe-webhook` backstops the browser. There is roughly a second between the card succeeding and step 4 running; if the tab closes in that window the money is taken and the contest never goes live. Stripe retries for up to three days independently of any browser, so the webhook completes those payments. It does not reimplement anything — it verifies the signature, then calls `confirm-launch` with the same payload the browser sends, using the service role key. confirm-launch's `if (contest.paid) return` guard makes the pair idempotent, so whichever arrives second is a no-op and no duplicate receipt is sent. Deliberately additive: the existing path is unmodified, so a broken webhook can only leave things as they were.

Test → live switch: replace `STRIPE_SECRET_KEY` (Supabase secret) and `VITE_STRIPE_PUBLISHABLE_KEY` (Vercel env + redeploy) with live keys from the same Stripe account. Never paste the live secret key into chats or docs.

---

11. Contest lifecycle and email

States: `draft → submission → voting → closed` (+ terminal `cancelled`). Cron `advance-contest-phases` (every minute) moves on `submission_ends_at` / `voting_ends_at`. The notify trigger (0013, extended in 0023) fires on three transitions: entering `voting`, entering `closed` with no winner yet (creator only — the cue to pick), and `winner_submission_id` being set. Each has its own dedupe stamp (`notified_voting_at` / `notified_closed_at` / `notified_winner_at`) written before sending, so a status flip-flop can't re-email anyone.

Senders: all outbound = `noreply@namingcontest.com` (Resend domain-verified; DKIM on `resend._domainkey`, SPF/MX on `send.` subdomain). `hello@` is the inbound human channel only — **it has no mailbox until the client sets up eNom forwarding**; until then contact-form mail bounces. DMARC is `p=none`; tighten to quarantine after reviewing reports. Root SPF record: none (nothing sends from the root domain). Deliverability (Promotions-tab placement) is reputation-driven and improves with real volume.

Subjects: `Your contest is live — {name}`, `Your vote is needed — {name}`, `Time to crown the winner — {name}`, `The winning name — {name}`, `Your name won — {name}`, `Contact form — {topic} — {name}`, `We got your message — NamingContest`.

Fast-forwarding a contest for testing: `docs/TESTING-EMAILS.md`.

---

12. Demo/mock system (important context)

The app began as a front-end-only prototype. Remnants, deliberately kept:

- `/v4/map` and `/v4/demo` are **unrouted** (404) but `PlatformMap.jsx`, `DemoIndex.jsx` and `src/data/v4/mock*` remain in the tree.
- Real pages branch on `mockContest` in ~150 places across 14 files (30 in ContestManage alone). Removing the mock plumbing is a real refactor with regression risk on pages that need live session state to test — scoped out before handoff on purpose. Tag `demo-complete-2026-07-20` restores the routed demo if ever wanted.
- `src/pages/*.jsx` outside `v4/`, `legal/`, `system/` are the legacy v1 prototype (BriefBuilder etc.). **Unrouted** — they were live in production serving mock data as though real, and `/docs` additionally advertised $9/$29/$89 tiers that never shipped. Removing their imports cut the bundle from 2,800 kB to 1,726 kB. The files remain; deleting them is a safe cleanup-pass candidate.
- Some files are oddly named (`PartnerSimulator`, `LegalCrumbs`) to dodge ad-blockers — do not rename back.

---

13. Known gaps / recommended review focus

1. `readSetup()` fallbacks vs. real sessions (see §9) — the historic bug source.
2. Social handles in email footers are **placeholder accounts nobody owns** (`x.com/namingcontest` etc., defined in `_shared/email.ts` SOCIAL const).
3. Landing testimonials are fictional; swap before launch.
4. Rich link previews: SPA serves one static `index.html`, so shared reveal links get a generic OG card. Fix = edge function serving crawler HTML.
5. Voting limit is hard-coded at 3 (DB trigger + UI clamp); the brief's `votingLimit` setting is vestigial.
6. `hello@` mailbox missing (client task, eNom forwarding).
7. Beta gate is client-side only.
8. No automated tests; the build (`npm run build`) is the only gate.
9. Bundle is one ~2.8 MB chunk — code-splitting would help mobile.

---

14. Operational runbook

```
-- Recent contests
select id, working_name, status, paid, voter_tier, created_at
from contests order by created_at desc limit 20;

-- Everything-applied check (all seven true)
select
 (select prosrc like '%submissionLimit%' from pg_proc where proname='enforce_submission_rules') as m0016,
 (select count(*) from pg_trigger where tgname='contests_set_price')=1      as m0017,
 (select count(*) from pg_proc where proname='get_ballot')=1                as m0018,
 (select count(*) from pg_policies where tablename='submissions'
    and policyname='submissions_read' and qual like '%creator_id%')>0       as m0019,
 (select count(*) from pg_proc where proname='rate_limit_take')=1           as m0020,
 (select count(*) from pg_trigger where tgname='contests_guard_payment')=1  as m0021,
 (select count(*) from pg_trigger where tgname='participants_tier_cap')=1   as m0022;

-- Did the DB actually call the notify function?
select id, status_code, left(content,300), created
from net._http_response order by id desc limit 5;

-- Cron health
select jobname, schedule, active from cron.job;
```

Function logs: Supabase dashboard → Edge Functions → function → Logs. Email deliveries: Resend dashboard. Payment records: Stripe → Payments.

---

15. Key rotation (when needed)

- `RESEND_API_KEY`: create new in Resend → `supabase secrets set` → done (functions read at runtime, no redeploy).
- `NOTIFY_SECRET`: generate → `vault.update_secret` for the DB side AND `supabase secrets set` for the function side. Both, or notify 401s.
- Stripe keys: dashboard roll → secret via `supabase secrets set`, publishable via Vercel env + redeploy (build-time inlined).
- Supabase anon key: dashboard → Vercel env + redeploy.